TOS Network

Open Coordination and Settlement for AI Services and Physical Edge Intelligence

TOS Network

2026 Edition

Abstract

TOS Network is being built as an open coordination and settlement network for autonomous agents, owner-operated AI services and site-bound physical AI terminals. Its economic unit is a completed service action under explicit policy, not an hour of unidentified hardware. Providers retain control of their models, devices, sites, data and operational responsibility; the network is intended to standardize identity, discovery, authorization, service commitments, evidence and settlement across organizational boundaries.

The implemented foundation, TOS Core, is an actor-model blockchain with native TVM execution, value-bearing asynchronous messages, masterchain-rooted consensus, dynamic sharding and ADNL/DHT/RLDP networking. It includes foundations for Agent Accounts, concurrent Service Actors, Task Escrow, Dispute, Capability Registry and Proof Attestation. It also contains a feature-gated Artificial Intelligence Proof of Work (AIPoW) path: settled-work indexing, bonded score commitments, public challenges, epoch settlement, Merkle distribution, reward maturation and deterministic masterchain mint validation. That path is inactive by default and creates no TOS unless governance activates it after the launch gates in this paper. The broader service protocol, ARD-compatible discovery products and AI Edge Computing Terminals described in this paper are planned product layers and are not presented as already deployed commercial infrastructure. TOS adopts Agentic Resource Discovery (ARD) rather than defining an incompatible general-purpose agent catalog, and the planned off-chain protocol product can run an independently deployable, federated ARD Registry. Models, prompts, sensor streams and private customer data remain off-chain; shared authority, commitments and settlement use the chain where independently operated parties require a tamper-resistant source of truth.

Contents

1 Executive Summary

TOS Network's long-term objective is to function as the Internet Layer for Autonomous Agents: a shared identity, discovery, authorization and settlement substrate that lets independently owned agents and AI-capable devices find one another, establish trust, exchange capabilities and settle value, in the same way TCP/IP and DNS let independently owned computers interoperate without a single owner. The remainder of this paper describes the concrete protocol and product work required to reach that objective, and is explicit about what is already implemented versus planned.

AI is moving from centralized applications toward autonomous software and independently operated model services. At the same time, inference is moving toward cameras, robots, vehicles, factories, stores and homes where latency, privacy, intermittent connectivity and physical safety matter. This creates a fragmented economy of agents, models, devices, data, tools and human services. The resources can perform useful work, but they do not share a neutral system for persistent identity, machine-readable capability, bounded authorization, payment, receipts, evidence and dispute resolution.

TOS Network is designed to provide that common trust and economic layer. It does not compete to train foundation models, operate a centralized cloud or rent bare accelerators. At the application-network layer, it follows an asset-light coordination model: hardware, models, sites and operations remain with providers, while protocol participants standardize how services are described, invoked, evidenced and settled.

The architecture has five layers:

1. **TOS Core.** Native actor execution, consensus, sharding, identity primitives, payment and authenticated networking.
2. **TOS Service Protocol.** Versioned descriptors, authentication, quotes, payment authorization, receipts, evidence envelopes and profile negotiation.
3. **Owner-operated terminals.** Bounded local execution through approved service adapters without exposing a public shell, container socket or raw accelerator.
4. **ARD discovery and clients.** Standard domain-hosted catalogs, independently deployable federated ARD Registries, human-readable names, policy-aware selection and verifiable handoff to TOS descriptors.
5. **Vertical profiles.** Managed AI, physical AI, storage, commerce and human services that share common protocol foundations but retain distinct state machines.

Only the first layer is substantially implemented in this repository. The remaining layers are a product architecture and delivery plan. This distinction is deliberate: validators do not execute application workloads, and a new model, accelerator or service profile must not require a consensus upgrade. The AIPoW contract and native-issuance path are implemented inside the first layer but remain feature-gated and inactive; the production scoring methodology, independent scorer reproduction, external audit and governance activation remain launch gates rather than implied production facts.

ARD operates before invocation: it lets agents publish and find resources. TOS remains responsible for authentication, live quotes, local admission, bounded execution, metering, evidence, receipts, payment and settlement. A search result therefore never becomes spending, execution, update, fleet or physical-control authority.

1.1 Strategic Positioning

The unit of value in TOS Network is a completed, policy-compliant service action. A consumer requests a capability with an exact service revision, input and output contract, price, deadline,

region, data-egress policy and evidence level. An owner-operated terminal decides whether it can safely admit the request, executes it through an approved local runtime, returns a signed receipt and settles according to the agreed payment policy.

For physical terminals, the value is often location rather than raw throughput. A low-power terminal beside a production line, camera or robot can provide low-latency and privacy- preserving execution that a remote cloud cannot provide economically or safely. Local safety and deterministic real-time work always take priority, and TOS never replaces an independent safety controller.

1.2 Economic Wedge, Flywheel and Falsifiable Metrics

The initial wedge is narrower than "a decentralized market for all AI." It is a managed, owner-operated terminal that turns a known model or tool into a versioned service with a live quote, bounded authorization, signed receipt and settlement. The protocol can then reuse the same identity, evidence and payment path for additional services without putting their application logic into consensus.

If the wedge works, the intended flywheel is:

`unrelated paid demand -> verified settlement history -> bounded AIPoW subsidy` $\longrightarrow$ `broader capable supply -> better locality, availability and price` $\longrightarrow$ `more unrelated paid demand`.

The defensible asset in this loop is not token issuance by itself. It is the accumulated, portable graph of service identities, exact capability revisions, payer-provider settlement, evidence, disputes and operating reliability, together with terminal integrations that can execute those services safely. Because discovery is open and federated, this advantage must come from better transaction completion and trust data rather than from locking providers inside a closed catalog.

The loop is falsifiable. Before describing it as a network effect, public reporting should show at least: organic settled value and its share of all settled value; unrelated unique and repeat payer control domains; paid completion and dispute-overturn rates by capability and evidence tier; AIPoW issuance divided by organic settled value; provider retention and non-reward service revenue; latency and availability by compute tier; and the top-one, top-four, top-ten and Herfindahl concentration of both settled value and AIPoW score after common-control aggregation. Subsidized circular volume, address count and declared capacity are not substitutes for these measures. Section 12.6 defines the demand and reward quantities used by this reporting.

1.3 Scale of the Opportunity

If autonomous software continues to multiply into the trillions of independently operating agents, machine-to-machine service requests become an economic layer in their own right, additive to and distinct from existing human-facing digital commerce. Under that trajectory, the addressable market for agent-to-agent and agent-to-terminal service transactions could reach into the trillions of dollars over time. TOS Network's design goal is to provide the identity, authorization, evidence and settlement infrastructure this layer requires. This is a structural thesis about where machine-native demand could go, not a claim about currently realized transaction volume, protocol revenue or token appreciation.

1.4 Model Scale and the Coming Deployment Shift

A useful way to read where machine-native demand will concentrate is to cross model capability against where that model actually runs. Provider roadmaps and open-weight release patterns point

toward a rough correspondence between model scale, deployment location and workload:

Model scale	Primary deployment location	Typical workload
2B–4B	Phones, IoT devices, single-board computers	Voice, control loops, sensor interpretation
8B–14B	Edge nodes: workstations, AI PCs, Jetson-class appliances	Autonomous agents, coding assistance, office and knowledge work, retrieval-augmented generation, tool calling
30B–70B	Enterprise servers	Enterprise knowledge bases, complex multi-step reasoning
200B+	Cloud data centers	Frontier research, large-scale training

Under this trajectory, the 8B–14B band is the most likely candidate to become the dominant model scale for edge AI over the next several years: it is small enough to run on owner-controlled hardware with acceptable latency and cost, and capable enough to carry agentic workloads such as tool calling, coding and retrieval-augmented reasoning that previously required a frontier-scale model.

Several factors reinforce this band as a practical sweet spot rather than an arbitrary midpoint. Local execution keeps round-trip latency low and avoids dependence on external network availability, which matters for interactive agent loops and tool-calling chains. Running on owner-controlled hardware keeps prompts, retrieved context and tool outputs on-premises by default, which is materially different from a privacy posture that depends on a remote operator's data-handling policy. A model in this band can run continuously within consumer or small-business power and cooling budgets, and it can keep serving previously approved workloads through a network outage. None of this implies that an 8B–14B model matches frontier-scale reasoning quality; it implies that, for the volume of narrow, tool-mediated and retrieval-grounded tasks autonomous agents generate, this band is frequently sufficient, and materially cheaper and faster than routing every request to a frontier model.

This reshapes what "the Internet Layer for Autonomous Agents" actually connects. The counterparties TOS Network coordinates at scale are less likely to be a comparatively small population of data-center accelerators, and more likely to be a much larger population of owner-operated edge nodes running 8B–14B class models. Because these nodes combine local inference with local storage, tool invocation and direct access to local data, they are complementary to centralized cloud compute rather than a substitute for it: cloud data centers remain the venue for frontier training and the heaviest reasoning workloads, while edge nodes carry the high-frequency, latency- and privacy-sensitive agent work. This is a structural thesis about where deployable model capacity is heading, not a commitment to specific hardware targets or a claim about the number of nodes currently connected to the network.

1.5 From Cloud-Centric AI to an Agent Mesh

The deployment shift described above changes the shape of the network TOS coordinates, not merely its scale. A cloud-centric pattern routes every request from a user directly to a centralized frontier model:

```
User -> Cloud Frontier Model
```

An edge-capable pattern instead resolves a request through progressively broader compute tiers, escalating outward only when a nearer tier cannot satisfy the required capability, latency, privacy or evidence policy:

```
User -> Local AI Agent -> Nearby Edge AI Node -> Regional AI Infrastructure ->
                                Cloud Frontier Model
```

These are compute tiers defined by device capability and network locality, not the five protocol layers introduced earlier in this section: TOS Core, the Service Protocol, a terminal, ARD discovery and a vertical profile can each be exercised at any tier, from a phone running a local agent to a data-center cluster running a frontier model. Mobile AI (phones, wearables and other constrained devices), Edge AI (owner-operated workstations, AI PCs and Jetson-class appliances), Regional AI (enterprise and aggregation-tier servers) and Cloud Frontier AI (large data-center deployments) correspond to the model-scale bands in the table above.

A single agent-mediated task can touch more than one tier. A local agent may handle routine tool calls and context assembly itself, delegate a specialized sub-task to a nearby edge node with a particular model or tool integration, and escalate only the narrow portion of the task that requires frontier-scale reasoning. Section 7.5 describes how the service protocol resolves a request across tiers without granting a search result execution or spending authority.

TOS Network does not select which tier is correct for a given request, and it is neutral to which foundation model, open-weight release or proprietary provider a node runs. Its function is the same at every tier: identity, capability description, authenticated discovery, quoting, bounded authorization, evidence and settlement.

As capable open-weight models proliferate across consumer and small-business hardware, the long-run edge-tier device population could plausibly reach into the hundreds of millions to billions of devices. This is a structural expectation about where deployable compute is heading, not a claim about the number of devices currently connected to the network or a specific forecast date.

2 Design Principles

2.1 Service Outcomes, Not Bare Capacity

Consumers buy a versioned service capability and a declared outcome. Hardware declarations, TOPS, accelerator memory and benchmarks are scheduling evidence; they are not themselves the service contract. Bare GPU rental, public accelerator access and arbitrary consumer-supplied programs or containers are outside the TOS product plan.

2.2 Actor First

Every account is an actor. Agent Accounts, Task Escrows, capability entries, service actors and verifier actors are ordinary native accounts with isolated state. There is no privileged global application process.

2.3 Asynchronous by Default

Cross-actor workflows use messages, callbacks, deadlines and retries. A sender cannot assume that another actor executes synchronously or that a multi-step workflow is atomic. Contracts must expose explicit intermediate states and deterministic failure behavior.

2.4 Authority Must Be Explicit

Owner, controller, creator, assigned agent, verifier, service and fee-payer roles are distinct. Every transfer or state transition must identify its authority. Off-chain runtimes are not trusted merely because they operate an agent.

2.5 Verification Without Payload Bloat

The chain is authoritative for funds, permissions, deadlines and settlement. Large prompts, model outputs, datasets and private credentials remain off-chain. Contracts store hashes, signatures, attestations and evidence references that bind external work to on-chain state.

2.6 Local Control and Safety

Providers retain custody of hardware, data, models and operational policy. A physical terminal continues approved local work while disconnected and keeps blockchain consensus outside the hard real-time and actuator-control loop. Raw sensor and physical-I/O interfaces are not public network capabilities.

2.7 Bounded Resources and Recoverable Failure

Connections, queues, retries, model caches, RAM, accelerator memory, durable journals and watchers must have explicit limits and cleanup paths. Failure, cancellation, restart, disconnect and payment rejection are normal protocol states. No anonymous or unpaid caller may cause unbounded memory, disk or durable-state growth.

2.8 Native Execution Focus

TOS Core has one active application execution domain: the native TVM basechain. Unsupported execution engines are not registered in the current product. Application workloads run in separate terminal processes and repositories, not inside `validator-engine`.

2.9 Open Discovery Compatibility

TOS uses the open Agentic Resource Discovery specification for protocol-neutral resource publication and search. TOS-specific payment and execution semantics are referenced from ARD records rather than encoded as incompatible replacements for ARD. Support is pinned to an exact assessed specification version, and changes to the external draft require compatibility review and new conformance vectors.

3 System Architecture

3.1 Layer and Repository Boundaries

TOS Network is designed as independently released layers. The separation limits consensus risk and allows product profiles to evolve at application speed.

Layer	Responsibility	Delivery boundary
TOS Core	Consensus, TVM, generic contracts, query APIs, wallet and cryptography, DNS, ADNL/DHT/RLDP and TOS Sites	Implemented and maintained in the `tos` repository
Base service plane	ARD profile and Registry, descriptors, manifests, authentication, quotes, payment authorization, receipts, evidence, Edge Core and conformance	Planned separate `tos-service-protocol` repository
AI terminals	General and physical terminal products, resource probes, runtime adapters, scheduling, signed updates and fleet management	Planned separate `tos-ai` repository
Other profiles	Storage leases, commerce, fulfillment and human-service state machines	Planned independently released vertical repositories

Existing Agent, Service, Escrow, Registry and Attestation contracts remain reusable TOS Core foundations. Product-specific daemons, model runtimes, catalogs and continuous device telemetry do not belong in the validator repository.

3.2 Masterchain and Basechain

The masterchain anchors validator sets, consensus configuration, workchain descriptors and cross-shard finality. The basechain executes native TVM accounts and contracts. The canonical genesis registers the masterchain with identifier -1 and the native basechain with identifier 0.

The masterchain does not execute an AI model. It establishes the shared state required for agents and task actors to trust message delivery, account state and settlement outcomes.

3.3 Accounts as Actors

An account transition can be represented as

$$(S_{t+1}, M_{out}) = F(S_t, M_{in}, C_t),$$

where S_t is the account state, M_{in} is one inbound message, C_t is the consensus execution context and M_{out} is a finite set of outbound messages. The transition changes only the receiving account. Effects on another account occur later when an emitted message is delivered there.

This isolation maps directly to AI systems: planners, workers, tools, tasks and verifiers can progress independently while retaining a shared, auditable settlement layer.

3.4 Cells and TVM

TOS stores account code and persistent state in immutable cells. Cells form directed acyclic graphs and are addressed by cryptographic hashes. TVM executes contract code deterministically, charges

gas and emits messages. Compact cell-native state is well suited to balances, policy, status values and content commitments.

3.5 Asynchronous Messages

Messages carry a destination, optional source, value and body. Internal messages have an unforgeable source account. External messages may carry signatures and replay-protection data. Value transfer is therefore part of the same mechanism as task acceptance, service calls and settlement.

AI workflow messages should include an opcode, a correlation value such as `query_id`, clear value semantics and enough domain binding to prevent replay against another account, task or network.

3.6 Sharding

TOS follows a bottom-up sharding model. Logically, each account has an independent history; physically, many account histories are grouped into shardchains. Shards may split and merge as load changes. Cross-shard messages preserve the actor abstraction even when communicating accounts reside in different shards.

This model permits parallel execution because transactions affecting different destination accounts do not mutate shared application state. High-volume agent economies can distribute tasks and services across many accounts instead of concentrating all workflow state in one contract.

3.7 Consensus and Networking

Validators agree on masterchain and shardchain blocks and enforce protocol configuration. The networking stack provides authenticated datagram transport, distributed discovery, reliable large-message delivery and overlay communication. ADNL identities and TOS DNS can locate services; RLDP and TOS Sites can carry application traffic. These primitives do not by themselves provide the planned service manifest, quote, receipt, relay or terminal product. `validator-engine` remains the canonical node process; `tosctl` provides an operator path for configuration and existing agent-oriented workflows.

4 Trust and Settlement Actors

4.1 Agent Wallet

An Agent Wallet is the custody and local-profile layer for an autonomous agent. The local profile records the underlying native wallet, owner key reference, controller key reference, spending policy, capabilities, optional metadata commitments and optional runtime binding.

Creating a profile generates separate owner and controller keys in the configured secrets vault. Funding transfers TOS to the derived native wallet address. Activation deploys the underlying wallet contract after the address has sufficient balance.

The owner retains full custody authority. Manual owner transfers and emergency operations are operator actions. Automated agent spending must use the policy-enforced Agent Account path. The owner key must never be exported to an off-chain agent runtime.

4.2 Agent Account

An Agent Account is the on-chain authority boundary for automated actions. Its state includes:

- owner address and controller public key;
- sequence number and expiry checks for replay protection;
- maximum value per controller action;
- cumulative daily limit and current daily spend;
- default task timeout;
- optional metadata and service-endpoint hashes.

Owner-authorized internal messages update policy or rotate the controller. Controller-signed external messages may send bounded task actions only after signature, expiry, sequence, per-action and UTC-day checks succeed. Local CLI checks improve operator feedback, but the contract is the final authorization boundary.

4.3 Task Actor

A Task Escrow is an account representing one unit of work. It stores the creator, optional assigned agent, optional verifier, budget, deadline, review period, lifecycle status, result hash, evidence hash, settlement-policy hash, permission hash and dispute hash.

The implemented lifecycle includes:

```
Open -> Accepted -> Submitted -> Settled
```

with terminal or review paths for rejection, cancellation, expiry and dispute resolution. An unassigned open task may be claimed atomically; the first valid claim records the caller as the agent. Result submission starts a review window. Authorized settlement releases a bounded payout and returns any remainder according to contract rules.

4.4 Capability Registry Actor

A Capability Registry entry is one contract per registered agent or service. It is not a chain-wide dictionary and does not by itself provide complete network enumeration. Its state commits to:

- owner and optional verifier;
- active or inactive status and registration time;
- task-category, pricing, metadata and verification-method hashes;
- a stakeable bond;
- an accumulating signed reputation score.

Owner-authorized operations update metadata, rotate the verifier, withdraw bond, deactivate or reactivate the entry. Staking is permissionless. Reputation updates require verifier authority. The registry exposes inspectable commitments while allowing detailed service descriptions to remain off-chain.

4.5 Service Actor

A Service Actor represents a bounded model, data, tool or other defined service. The implemented contract supports open access or one authorized caller, price per call, a UTC-day rate limit, metadata and proof-scheme hashes, request and response commitments, accumulated revenue and an active or inactive lifecycle. A call must attach at least the configured price; accepted value is credited to contract revenue. The owner commits response hashes, updates policy, withdraws revenue and controls activation. The registry says what a service claims to provide; the Service Actor governs calls and payments. It does not expose a host, raw accelerator or arbitrary execution environment.

4.6 Verifier and Attestation Actors

A Verifier Actor evaluates result or evidence commitments according to a declared policy. A verifier may accept, reject, score or dispute work, but its authority must be explicitly stored by the task or registry actor. The implemented Proof Attestation foundation can record compact claims and evidence references. Neither a payment nor a hash alone proves semantic correctness, confidentiality or the physical hardware used. TOS does not hard-code one model, oracle or proof backend.

5 Agent Wallet and Authority Model

5.1 Owner and Controller Separation

The owner controls custody, recovery and high-authority policy changes. The controller is an automation key with bounded authority. Compromise of a controller should not become owner-equivalent compromise.

For a controller action with value v, per-action limit L_a, daily limit L_d and already spent amount D, the Agent Account requires

$$0 < v \leq L_a \quad \text{and} \quad D + v \leq L_d.$$

It also verifies a valid signature, an expected sequence number and a non-expired timestamp.

5.2 Operational Lifecycle

The initial operator lifecycle is:

1. create a local Agent Wallet profile and vault keys;
2. fund and activate the underlying native wallet;
3. build and inspect deterministic Agent Account state;
4. deploy the Agent Account through an active funding wallet;
5. bind an off-chain runtime to the profile;
6. execute bounded task actions with the controller;
7. use the owner for policy updates, controller rotation and explicit custody actions.

5.3 Policy and Permission Linkage

Task actions can bind a locally tracked permission identifier to an on-chain permission hash. Before signing, tooling checks the target contract, lifecycle state, assignment and permission linkage. The Task Escrow independently validates sender authority and state transition rules. This layered design catches operator mistakes without treating local configuration as chain authority.

6 Task Markets and Settlement

6.1 Creating Work

A creator deploys a Task Escrow with a budget, deadline, settlement policy and optional agent or verifier. The deployment value must cover the intended escrow and execution costs. Task metadata itself may be stored off-chain; the contract retains compact commitments.

6.2 Assignment and Claim

A task can target a specific Agent Account or remain open. A targeted agent may accept or reject according to contract state. An open task may be claimed atomically. Assignment is authoritative only after it appears in chain state.

6.3 Results and Evidence

The agent submits a result hash and evidence hash. These values do not prove semantic quality by themselves; they bind a later-retrieved artifact or attestation to the task. The review period gives the creator or verifier time to settle or dispute the submission.

6.4 Settlement and Disputes

Settlement transfers no more than the available escrow. Cancellation and rejection refund according to explicit lifecycle rules. Timeout is consensus-time based. A dispute records a reason or evidence commitment and moves authority to the configured resolution path. Verifier resolution is bounded by contract balance and role checks.

7 TOS Service Protocol and Discovery

7.1 Base Protocol

The planned TOS Service Protocol is the common application plane above TOS Core. Independent implementations require canonical and versioned definitions for:

- signed service descriptors and short-lived terminal manifests;
- profile negotiation and handling of unknown critical extensions;
- authentication, replay domains, nonces and bounded delegation;
- quotes, payment authorization, cancellation, refunds and idempotency;
- request, stream, result, receipt and evidence envelopes;
- durable operation identities and restart recovery;
- error classes, limits and compatibility rules.

This protocol is planned work, not functionality implied by the existence of a Service Actor contract. The contract is a settlement primitive; an interoperable session and receipt protocol still requires schemas, SDKs, conformance vectors and independent implementations.

7.2 Agentic Resource Discovery Compatibility

TOS adopts Agentic Resource Discovery (ARD) as the public, protocol-neutral discovery envelope for agents, APIs, tools, workflows and nested catalogs. The initial compatibility target is ARD v0.9 Draft as assessed on July 31, 2026.[1] Because that document is a proposal rather than a frozen standard, every implementation records the exact version it accepts and does not silently reinterpret changed fields.

An operator with a publicly verifiable DNS name publishes the catalog at `https://publisher.example/.well-known/ai-catalog.json`, replacing the example host with its FQDN. Each resource uses the ARD domain-anchored identifier form `urn:air:<publisher-fqdn>:...` and an exact value-or-reference and media-type representation. An entry can hand a client to MCP, A2A, OpenAPI, a nested catalog or a versioned TOS Service Protocol descriptor. The ARD catalog is

[1] Agentic Resource Discovery specification: `https://agenticresourcediscovery.org/spec/`.

stable discovery metadata; rapidly changing price, free capacity, queue depth, local safety state, model residency and availability are returned only by a fresh TOS quote and admission flow.

TOS can run a separately deployable ARD Registry. It implements the mandatory versioned HTTP REST search baseline, including `POST /search`, and may ingest public catalogs, operator-approved private catalogs, upstream registries, TOS on-chain discovery seeds and signed profile manifests. Registry responses preserve whether each field came from the publisher, on-chain state, live observation, attestation or registry-derived ranking. Federation is plural: no single TOS Registry is mandatory, and referral depth, cycles, fan-out, time, response size, caching and retries are bounded.

ARD stops before invocation. It does not define TOS authentication, authorization, scheduling, execution, payment, receipts, evidence or settlement. Before transferring value or invoking a service, a client independently verifies publisher binding, the current TOS descriptor, runtime and endpoint authorization, manifest commitments, chain activity, quote expiry, admission and payment destination.

7.3 Capability Discovery

Capability discovery separates compact on-chain commitments from extensible off-chain descriptions. A client can inspect an entry's owner, activity, bond and commitment hashes, then retrieve a signed descriptor and verify its relationship to chain state.

The current per-actor registry does not solve chain-wide search. TOS ARD Registries enumerate and rank services while retaining the Capability Registry as one source of on-chain seeds and commitments. Indexed data is a derived view: clients must verify critical publisher, owner, authorization, capability, activity, endpoint, quote and payment fields against signed, observed or on-chain state before committing funds.

7.4 Names and Reachability

A service may be reached through a human-readable `name.tos` identity or a raw ADNL identity. A descriptor may publish RLDP/TOS Sites and optional HTTPS, QUIC, WebSocket or streaming bindings without changing the identity and authorization model. Ordinary home and site networks may require an owner-selected relay or reverse tunnel. A relay may forward encrypted traffic but must not receive owner authority, unrestricted runtime credentials or settlement control.

A public root `.tos` registration product, signed multi-endpoint descriptors, short-lived health data and production relay service are planned. DNS and TOS Sites primitives already exist, but they are not equivalent to a complete service discovery marketplace.

A `name.tos` identity is not automatically a conventional public-DNS trust anchor for ARD. A public service without its own DNS domain uses an approved HTTPS gateway that binds an ARD publisher namespace to the separately signed TOS identity, or it participates in a private Registry with an explicit `.tos` resolution policy. ARD publisher control, `name.tos`, ADNL, account and manifest commitments remain distinct verifiable bindings.

7.5 Locality-First Routing Across Mesh Tiers

The mesh tiers introduced in Section 1.5 are realized through the discovery and invocation primitives described earlier in this section, not through a separate escalation protocol. A client or agent runtime performing an ARD search can rank candidate resources by measured or declared latency, region and privacy policy in addition to price and capability, and a federated ARD Registry can apply the same ranking when resolving a query across upstream catalogs. Preferring the nearest capable

node before considering a broader tier is a client- or registry-side policy expressed through existing search, ranking and federation behavior, not a new consensus primitive.

A single task may fan out across tiers before it settles. An agent running locally may resolve routine sub-tasks itself, issue independent quotes to one or more nearby specialized edge nodes for narrow capabilities such as retrieval, transcription or a domain-specific tool, and reserve a regional- or frontier-tier service call for the portion of the task that requires it:

```
User -> Nearest Edge Agent -> Nearby Specialized Agents -> Regional Compute ->
                                Cloud Frontier Model
```

Each hop in this pattern is an ordinary TOS-authorized invocation: a resolved descriptor, a live quote, a bounded payment authorization and a signed receipt. ARD discovery lets an agent find a candidate at any tier; it does not grant that candidate execution, spending or settlement authority, and a locality-first search ranking is evidence for the requester's own admission and payment decision, not a substitute for it.

8 AI Edge Computing Terminals

8.1 Terminal as the Product Unit

The TOS AI Edge Computing Terminal is the first planned owner-operated execution product. It combines measured local resources, approved runtime adapters, bounded task admission, TOS identity and reachability, metering and signed receipts. A terminal may be a Linux GPU workstation, an AI PC, a small edge server or an industrial ARM/Jetson appliance.

Consumers schedule against a service capability such as text generation, embedding, OCR, speech recognition or video analysis. A capability states the exact model or task revision, schemas, limits, privacy and region policy, evidence level, price and expiry. Hardware model numbers and vendor performance claims are evidence used by the scheduler; the consumer does not receive raw access to the device.

Consistent with the model-scale trend described in Section 1.4, the terminal product is designed first for the 8B–14B parameter class: the band most compatible with owner-operated hardware such as workstations, AI PCs and Jetson-class appliances, and the band most likely to carry agentic, coding, RAG and tool-calling workloads at the edge. The terminal architecture does not require this band and does not exclude smaller or larger models where a runtime adapter and resource envelope support them.

The set of eligible terminal hardware is intentionally broad within that class: a Mac mini, MacBook, gaming PC, home server, NAS appliance or small industrial workstation can register under the same Capability Registry and Service Actor primitives described earlier in this paper as a rack-mounted GPU server, provided it passes the same admission, isolation and evidence requirements described in this section. What a terminal contributes is not limited to raw inference: a Service Actor entry can equally represent local storage, request routing to another known-good node, retrieval or context-memory services built on stored state, or a narrowly scoped tool integration. The product requirement is a bounded, evidenced, receipted capability, not a particular hardware class or deployment scale.

8.2 Runtime and Executor Isolation

The public service listener is separated from the administrative interface. Public adapters run under dedicated operating-system identities, containers or equivalent sandboxes with explicit CPU, RAM,

accelerator-memory, filesystem, network and tool grants. They receive no Docker socket, shell, owner key, unrestricted wallet key, home directory or raw physical-I/O authority.

The operator approves every model and executable artifact in the initial product. A runtime adapter performs compatibility checks, bounded load and unload, cancellation, error normalization, usage reporting and cleanup after success, timeout, disconnect, out-of-memory failure or process crash. Consumer-supplied containers, native programs and models are not accepted by the TOS terminal product.

8.3 Admission, Scheduling and Resource Bounds

The local scheduler is authoritative for admission. It considers model compatibility, RAM, accelerator memory, storage, queue and in-flight limits, context and output bounds, deadlines, owner reservations, thermal policy, price, privacy and evidence requirements. Remote ARD discovery cannot reserve local hardware merely by publishing an optimistic capacity value. A physical-terminal catalog advertises a bounded site or fleet-broker capability, not raw per-device administration, site topology, sensor streams or actuator handles.

Every implementation must bound connections, sessions, streams, nonces, quotes, queues, retries, model caches, temporary files, logs, receipt queues, chain watchers and durable journals. Completion, cancellation, failed payment, adapter crash, upgrade and shutdown each need a tested cleanup path. These constraints are required to prevent anonymous RAM, VRAM, disk, watcher and durable-state growth.

8.4 Site-Bound Physical AI

A physical terminal runs beside sensors, robots, vehicles or industrial equipment. Its mandatory priority order is:

1. emergency and independent safety interlocks;
2. deterministic control deadlines;
3. local real-time perception and sensor fusion;
4. local asynchronous analysis and maintenance;
5. approved external service requests;
6. background update, telemetry and compaction.

TOS does not enter the hard real-time loop. Raw CAN, GPIO, serial, fieldbus, camera or actuator interfaces are never public capabilities. A narrow semantic action must pass local policy and an independent safety controller, which may reject an otherwise valid network request.

8.5 Disconnected Operation and Reconciliation

A physical terminal must continue its approved local workload while disconnected. It may use only unexpired cached policy and explicitly bounded offline authority. Events and obligations are written to a size- and age-bounded, tamper-evident journal. Reconnection first observes revocation and policy expiry, then uploads and reconciles events idempotently. A reconnect must not execute a paid task twice, lose a refund obligation or grow the journal without limit.

8.6 Safe Model and Software Updates

Model, runtime, firmware and policy updates are content-addressed, signed by an authorized release identity, compatibility-checked and staged before activation. Terminals maintain an active slot and a known-good rollback slot. Activation is crash-safe and subject to health gates, anti-rollback

policy and bounded artifact retention. Fleet rollouts use canary and progressive rings and stop automatically when declared health thresholds fail.

8.7 Fleet Management

Fleet state is primarily off-chain. A fleet service handles enrollment, scoped delegation, site grouping, staged rollout, health, revocation, offline expiry and retirement without publishing detailed topology or continuous telemetry on-chain. Commands have bounded fan-out, expiry and idempotency. Compromise of a fleet controller must not confer owner custody or override independent local safety policy.

9 Verifiable AI Workflows

9.1 State and Evidence Boundary

TOS records what consensus can enforce:

- account authority and spending limits;
- task assignment, status, budget and deadlines;
- settlement and dispute authority;
- result, evidence and metadata commitments;
- registry bond, activity and reputation references.

The chain should not store private prompts, service credentials, full model transcripts or large datasets. These remain in content-addressed storage or service infrastructure.

9.2 Proof Adapters

Verification may use signatures, attestations, proof-backed transport evidence, committee decisions or domain-specific proof systems. A proof adapter translates external evidence into a compact decision or commitment understood by a task or verifier actor. The core protocol remains neutral to the model provider and proof backend.

Claims must state their evidence level. A terminal declaration, an observed successful call, a standardized benchmark, an independent audit, hardware attestation and replicated execution are different forms of evidence and must not be presented as interchangeable. A signed receipt proves that a signer made a statement about an operation; its semantic meaning still depends on the accepted policy and trust model.

9.3 Managed-Service Workflow

1. A client resolves a service identity and verifies its signed descriptor.
2. The client requests a quote for an exact profile, limits and evidence policy.
3. A controller or user authorizes a bounded payment against the signed quote.
4. The terminal admits the request only after local policy and resource checks.
5. An isolated adapter executes the approved model and streams a bounded response.
6. The terminal signs a result and usage receipt linked to the operation identity.
7. The client verifies the receipt and the settlement path releases or refunds value.

Every step is asynchronous. Failure, timeout and dispute are explicit workflow states rather than hidden exceptions in an off-chain orchestration process.

9.4 Physical-Terminal Workflow

A site terminal receives sensor data locally, schedules safety and real-time perception ahead of all network work, applies local policy and emits a bounded event or service result. If the site is disconnected, it continues approved local operation and journals the event. After reconnection it observes revocations, reconciles the journal idempotently and publishes only the receipt or evidence allowed by site data policy. Blockchain availability is therefore not a prerequisite for an emergency stop or deterministic local control decision.

10 APIs and Operator Tooling

10.1 tosctl

`tosctl` is the canonical operator tool for node configuration, native wallets and AI actor workflows. The implemented command families create and inspect Agent Wallet profiles, deploy and manage Agent Accounts, create and operate Task Escrows, and deploy or update Capability Registry entries.

Human-readable output supports operators; JSON output supports agent runners and automation. Secrets remain in the configured vault rather than the profile JSON.

10.2 Read APIs

The authenticated node-control service exposes read endpoints without URI version prefixes:

- `GET /agents/{address}`;
- `GET /tasks/{address}`;
- `GET /tasks` with status, creator, agent, deadline and pagination filters.

Task-list chain reads use bounded concurrency, deterministic ordering, individual timeouts and failure isolation. The list enumerates tasks registered in the local node-control configuration; it is not a complete chain-wide index.

Public API errors distinguish invalid requests, missing state, unavailable RPC, invalid contract state and timeout without exposing raw internal transport errors.

10.3 Trust Tiers

An agent controlling material funds should verify state through its own node or a proof-backed client. A trusted remote API may be suitable for an operator's own infrastructure. Third-party indexers are useful for search and analytics but must not become authority for balances, permissions or settlement.

11 Security Model

11.1 Key Compromise

Owner keys belong in operator-controlled vaults. Controller keys belong to runtimes only when their on-chain authority is acceptably bounded. Rotation must be possible without changing the owner. Runtime manifests must never contain owner secrets.

11.2 Replay and Domain Confusion

Signed actions require sequence numbers and expiry. Protocols should bind signatures and permissions to the intended network, account, task and operation. A valid message for one workflow must not authorize another.

11.3 Escrow Safety

Contracts reject out-of-order messages, unauthorized senders and payouts above available balance. Deadlines, review windows and terminal states are explicit. Local CLI validation is not a substitute for contract checks.

11.4 Service and Evidence Risk

A capability claim is not proof of service quality. A metadata hash is not proof that its content is true. A result hash is not proof that a result is correct. Bonds, verifier roles, attestations and reputation can reduce risk, but task settlement policy must state which evidence is authoritative.

11.5 Executor and Supply-Chain Isolation

Public requests are untrusted input. Runtime adapters must enforce schema, byte, dimension, duration and output limits before allocating expensive resources. Models, runtimes and updates require authenticated provenance, canonical hashes, compatibility checks and operator approval. Sandboxing, least-privilege device access, network egress policy and secret separation limit the impact of a compromised model server or parser. The public service interface must never become an administrative or arbitrary-code execution path.

11.6 Physical Safety

Possession of a valid TOS key, payment authorization or network receipt does not grant raw actuator authority. Local semantic capability policy and independent safety interlocks remain authoritative. Safety and control deadlines pre-empt external work; overload or network failure must fail toward a locally defined safe state.

11.7 Offline Authority and Update Risk

Offline operation creates stale-policy and replay risk. Offline rights therefore need explicit scope, expiry and budgets, while journals need integrity, bounded retention and idempotent reconciliation. Update systems require separate release authority, anti-rollback checks, known-good recovery and staged fleet rollout. A terminal that cannot verify a package or recover a healthy runtime must continue only its previously approved safe workload.

11.8 Fleet Compromise

Fleet controllers use scoped delegation and revocable terminal identities. Commands carry target scope, expiry and idempotency identifiers. Rollout fan-out, retry queues, health history and offline inventory are bounded. Fleet compromise must not reveal owner custody keys, customer payloads or unrestricted site topology.

11.9 ARD Catalog and Registry Risk

Catalogs, descriptions, representative queries, endpoints, nested references and federated responses are untrusted input. Implementations validate the pinned ARD schema, publisher domain, resource identifiers, trust metadata, value-or-reference rules and media types. Natural-language fields are data, never instructions to an agent. Crawlers allow only approved schemes, ports, redirects and address classes; they defend against DNS rebinding, server-side request forgery, oversized or compressed input, recursive catalogs, federation cycles, stale rollback, equivocation and ranking manipulation.

Catalog, field, entry, reference, redirect, hop, response, time, retry, cache, index and per-publisher limits prevent anonymous memory, disk and work growth. Visibility policy prevents private endpoints, credentials, topology, sensor data and customer payloads from entering public search or federation. Registry output retains field provenance and cannot grant wallet, terminal, model, update, fleet or actuator authority.

11.10 Indexer Authority Drift

Search results and reconstructed timelines are non-authoritative. Critical clients should re-read relevant contract state before transferring funds, accepting work or resolving a dispute.

11.11 Availability and Message Amplification

Automated actors can create unbounded retry loops. Production workflows require funded and bounded retries, backoff, timeout policy, queue limits and observable failure records. Terminals additionally require soak tests for RAM, accelerator memory, disk, watchers, connections, offline journals and update artifacts across success and every failure path.

12 TOS Token Economics

12.1 Native Unit, Utility and Economic Character

TOS is the native settlement and security asset of TOS Network. One TOS equals 10^9 *nanotomi*, the smallest indivisible unit. Contract balances, fees, task budgets, bonds, spending limits and validator stakes are represented in *nanotomi* to avoid rounding ambiguity.

TOS is designed for functional use within the network:

- paying transaction, message, storage and protocol-service fees;
- bonding elected validators and securing consensus;
- compensating elected validators for producing and validating blocks;
- funding task escrow, refunds and service-call settlement;
- bonding capability, verifier and other protocol roles where specified; and
- participating in protocol governance only where an accepted protocol rule grants that function.

TOS does not represent equity, debt, a partnership interest or ownership of a legal entity. It gives no contractual claim on project assets, revenue, profits, dividends or cash flows; no right of redemption by a company, foundation or operator; no guaranteed return, fixed yield, liquidity or price support; and no ownership of a model, terminal, data set or off-chain service provider. Holding TOS alone does not produce a protocol reward. Validator compensation requires election, locked stake, active operation and continuing exposure to the protocol's complaint and penalty rules.

12.2 Supply Policy

The initial distribution is designed to avoid a large insider premine. Approximately 500 million TOS is created through ordinary validator block rewards; the remaining community-agent allocation (approximately 4.5 billion TOS of the five-billion total-supply policy) is created through Artificial Intelligence Proof of Work (AIPoW), a separate protocol reward mechanism described below, is not funded at genesis and is never held by a treasury wallet. The current production policy is:

Item	Policy target
Smallest unit	1 *nanotomi*
Unit conversion	1 TOS = 1,000,000,000 *nanotomi*
Approximate total network supply target	5,000,000,000 TOS
Approximate validator-reward creation target	500,000,000 TOS
Community-agent allocation (Artificial Intelligence Proof of Work)	4,500,000,000 TOS
Provisional main-wallet genesis balance	100,000 TOS
Elector operating reserve	500 TOS
Configuration-contract operating reserve	500 TOS
Provisional total genesis creation	101,000 TOS
Reference post-genesis validator creation	499,899,000 TOS
Proof-of-work giver allocation	0 TOS
Team, investor, foundation and treasury allocation	0 TOS
Target validator distribution duration	Approximately seven years
Treatment of network outages	No creation and no later catch-up
Reward termination	Configuration governance tapers or sets ConfigParam 14 to zero
New hard consensus supply cap	None in this policy

Five hundred million TOS of validator creation, five billion TOS of total supply and seven years are policy calibration targets, not an exact consensus cap, a guaranteed completion date or a statement about future market value. The actual total and completion date depend on finalized block production, shard behavior, validator availability, configuration activation and the eventual governance transaction that stops native block creation. The community-agent allocation is created only through Artificial Intelligence Proof of Work, described in its own subsection below, with its own launch gates; it is not distributed through ConfigParam 14 or the Elector.

12.3 Genesis Bootstrap and Allocation

The production zerostate provisionally creates 101,000 TOS: a bounded 100,000-TOS validator-bootstrap main wallet and 500-TOS operating reserves for each of the Elector and Configuration contracts. It contains no proof-of-work giver, faucet, discretionary mint contract, general treasury, market-making inventory or native allocation to a team, investor, foundation or ecosystem fund. The protocol distribution plan includes no sale of a large genesis inventory.

The zerostate commits four original validators with unique signing and network identities and equal initial consensus weight. The main wallet may transfer to each validator's published controlling masterchain wallet exactly 20,000 TOS of principal for two overlapping minimum-stake elections,

plus no more than 100 TOS of separately measured wallet, message and retry costs. The signing keys, controlling wallets, allowed destinations, transfer caps and transactions must be public.

After two overlapping elected validator sets have been installed successfully, all remaining main-wallet TOS must be transferred to a published unspendable address, leaving the wallet with zero spendable balance. The main wallet is a temporary bootstrap mechanism and must not become a treasury, fee collector, market-making account, governance fund or emergency mint authority. The two system-contract reserves are controlled only by their deployed contract code and are not validator rewards or discretionary allocations.

12.4 Validator-Led Native Creation

Post-genesis native creation uses the existing block-value and Elector path:

$$\begin{aligned} &\texttt{ConfigParam 14 block creation} \\ \longrightarrow\ &\texttt{configured collector or Elector fallback} \\ \longrightarrow\ &\texttt{active validator-set bonus pool} \\ \longrightarrow\ &\texttt{stake and bonus recovery} \end{aligned}$$

For validator i, the existing proportional rule is

$$B_i = \left\lfloor B_{\text{set}} \frac{E_i}{\sum_j E_j} \right\rfloor,$$

where B_{set} is the active set's recoverable bonus pool and E_i is validator i's effective stake. Deterministic integer-remainder handling, stake returns, complaints and penalties follow the existing Elector rules. There is no separate proposer allocation, passive-holder yield, work-score reward, vote-count reward, private mint key or off-chain reward discretion.

Penalties are proportional to the stake a validator controls rather than a flat amount. The misbehaviour schedule in `ConfigParam 40` sets a base fine that the severity and duration of the fault scale up, with the most serious tier reaching TM$2,500 plus one quarter of the stake. Duration thresholds are expressed as fractions of the validation round so that every tier is reachable within one. Two consequences are worth stating: an operator that gathers more stake carries proportionally more at risk rather than the same fixed exposure, and any contract holding stake on behalf of others can size the operator's own required funds from this schedule instead of guessing. A network that omits the parameter falls back to a flat fine with no proportional component, which makes the amount at risk independent of the amount controlled.

A validator must submit a valid bid, be selected, keep its stake locked, operate its assigned consensus responsibilities and remain eligible under the protocol rules. Native block creation and collected transaction fees are different economic flows: only ConfigParam 14 creation increases gross native supply, while fees transfer existing TOS.

12.5 Approximate Seven-Year Distribution

The reference post-genesis validator creation target is 499,899,000 TOS. For calibration, the expected creation rate is modeled as

$$\dot{S} = \lambda_{\text{mc}} R_{\text{mc}} + \sum_s \lambda_s \frac{R_{\text{bc}}}{2^{d_s}},$$

where λ_{mc} is the measured finalized masterchain block rate, R_{mc} is the configured masterchain creation value, λ_s is the finalized rate of basechain shard s, d_s is its prefix depth and R_{bc} is the configured basechain value. Depth adjustment is intended to keep aggregate basechain creation approximately stable across balanced shard splits, but actual finalized chain data remains authoritative.

The current implementation candidate uses 0.569879384 TOS per masterchain block and 0.335223167 TOS per unsplit-basechain block. At the locally measured reference rate of approximately 2.5 finalized blocks per second for each chain, those values project nearly the full post-genesis target from August 1, 2026 to August 1, 2033. They are provisional until sustained multi-validator, multi-host, outage and shard split-and-merge measurements pass the published launch gates.

Issuance is event-based:

$$\texttt{no finalized produced block} \Longrightarrow \texttt{no native creation}.$$

An outage, partition, failed election or delayed restart creates no emission debt. There is no backfill, catch-up multiplier, anniversary tranche, pending-emission queue or automatic extension of a seven-year calendar. Faster block production can reach the target earlier; slower production can reach it later. These outcomes must be measured and disclosed rather than described as protocol faults.

Governance must monitor finalized creation and begin a public taper review before projected gross validator creation reaches 495 million TOS. It may reduce ConfigParam 14 to manage activation delay and must set both masterchain and basechain creation values to zero near the 500-million validator-creation policy target. Transaction and service fees continue after native creation stops. Any later proposal to restart native creation is a separate monetary-policy decision requiring prominent public review.

12.6 Artificial Intelligence Proof of Work: The Community Distribution

The community-agent allocation of approximately 4.5 billion TOS is created through **Artificial Intelligence Proof of Work (AIPoW)**. The useful-work category is not new: Bittensor pays participants for validator-scored digital commodities, Gensyn focuses on execution and verification of machine-learning work, and Render operates a distributed GPU-work marketplace.[2] TOS does not claim that attaching a token to AI compute is itself an innovation. Its narrower design claim is that heterogeneous AI, storage, verification and physical-edge services can share one outcome ledger, and that bounded native issuance can be derived from unrelated, evidence-graded, on-chain settled demand rather than from declared capacity or a subjective popularity vote.

AIPoW borrows four economic properties from proof-of-work issuance: permissionless earning, protocol-automatic creation, no reward for merely holding a token, and no treasury that first receives the allocation. It does *not* replace Bitcoin's consensus work. TOS consensus is secured by stake-bonded validators; AIPoW provisions the service-capacity layer. This separation avoids asking non-deterministic useful work to provide block-timing, fork-choice and inexpensive universal-verification properties for which ordinary hash puzzles were designed. Those are known hard requirements when useful computation is proposed as consensus work.[3] The name AIPoW is therefore an issuance and service-provision analogy, not a claim that AI work secures TOS consensus and not a legal classification of any participant relationship.

[2] Primary project descriptions: https://www.bittensor.com/docs, https://docs.gensyn.ai/the-gensyn-protocol, and https://know.rendernetwork.com/.

[3] A. Merlina, T. Garrett and R. Vitenberg, "On Replacing Cryptopuzzles with Useful Computation in Blockchain Proof-of-Work Protocols," 2024: https://arxiv.org/abs/2404.15735.

12.6.1 Eligible Work and Evidence-Weighted Score

Let J_e be the set of unique service operations finalized in epoch e. For operation j, let p_j be its actual settled price in *nanotomi*, x_j its measured work units, r_{c_j} the published maximum rate per unit for capability class c_j, m_{ℓ_j} the evidence multiplier in basis points for evidence level ℓ_j, and $q_j \in [0, 10000]$ a capability-specific quality and service-level factor. Let I_j equal one only when the operation has a unique replay-protected identity, reached final settlement, remained outside dispute or forfeiture, used the frozen capability revision and rate card, and satisfies the published payer-independence and evidence rules. Its integer score contribution is

$$v_j = I_j \left\lfloor \min\bigl(p_j, r_{c_j} x_j\bigr) \frac{m_{\ell_j}}{10^4} \frac{q_j}{10^4} \right\rfloor .$$

Self-declared capacity has $m_{\text{declared}} = 0$. A paid receipt may establish that an operation was requested and settled, but it does not by itself prove semantic correctness. Higher evidence levels require capability-appropriate evidence such as a deterministic replay, randomized replicated execution, a benchmark with a hidden challenge seed, a hardware attestation bound to the service revision, an audited measurement, or a cryptographic proof. An LLM-as-judge score may be one input for an open-ended task but is not sufficient by itself for the highest evidence tier. Every multiplier and quality function is a bounded rational integer rule in the published methodology; floating-point or scorer-local behavior cannot affect an epoch root.

The price cap prevents a provider from turning an unusually high invoice into unlimited score, while the settlement cap prevents a rate card from crediting value no customer paid. It does not, by itself, prove that payer and earner are independent. Organic settled value is therefore a separate quantity:

$$O_e = \sum_{j \in J_e} I_j^{\text{organic}} \min\bigl(p_j, r_{c_j} x_j\bigr) ,$$

where $I_j^{\text{organic}} = 1$ only for a settled operation whose payer and earner are in different disclosed and inferred control domains, which is not a protocol-funded challenge, and which passes the frozen anti-wash-trading policy. Challenge work may earn a score under a separate allowance, but it never increases O_e.

12.6.2 Control-Domain Aggregation and Concentration

Addresses are aggregated before scoring. For control domain a, define raw epoch score

$$S_{a,e} = \sum_{j \in J_e :\, a(j) = a} v_j .$$

The published methodology then applies a concave tail and an absolute score cap:

$$\widehat{S}_{a,e} = \min\left\{ C_e, \begin{cases} S_{a,e}, & S_{a,e} \le H_e, \\ H_e + \lfloor \alpha_e (S_{a,e} - H_e) \rfloor , & S_{a,e} > H_e, \end{cases} \right\} \qquad 0 \le \alpha_e \le 1.$$

H_e is the full-credit threshold, α_e the published marginal factor above it and C_e the maximum credited score for one control domain. This is a cap on credited score, not a claim that one operator can never receive a large fraction of a thin epoch. Actual top-k reward shares and the Herfindahl index remain mandatory disclosures. Splitting addresses does not raise the limit when beneficial ownership, operator keys, payer relationships, hosting, network identity or other published common-control signals link them. Because hidden common control cannot be proven from chain data alone

in every case, the domain graph and each classification change are reproducible scorer inputs and are challengeable with the score root.

Let $\widehat{S}_e = \sum_a \widehat{S}_{a,e}$. The epoch Merkle tree commits leaves of the form $(a, \widehat{S}_{a,e})$, sorted by identity, together with the frozen methodology and rate-card hashes. Duplicate identities are invalid rather than silently merged.

12.6.3 Demand-Coupled Epoch Pool

The native epoch pool uses exact integer arithmetic. Let K_n/K_d be the configured emission coefficient, F_e the cold-start floor, U_e the published per-epoch schedule ceiling, C_{AI} the cumulative AIPoW cap and $M_{<e}$ cumulative prior AIPoW creation. The candidate pool and final mint are

$$P_e = \min\left(U_e, \max\left(F_e, \left\lfloor O_e \frac{K_n}{K_d} \right\rfloor\right)\right), \qquad M_e = \min(P_e, \max(0, C_{\mathrm{AI}} - M_{<e})).$$

This is the arithmetic implemented by the feature-gated native mint path. A positive cold-start floor is an explicit, measurable acquisition subsidy, not organic demand, and must be reported separately. Governance may set it to zero. A randomized challenge allowance in the scoring methodology is bounded by

$$S_e^{\mathrm{challenge}} \le F_e^{\mathrm{challenge}} + \left\lfloor O_e \frac{\chi_n}{\chi_d} \right\rfloor,$$

where the first term is a published cold-start allowance and χ_n/χ_d is the configured challenge multiplier. Related-payer and challenge settlements remain excluded from O_e, so they cannot recursively expand the demand-coupled pool. If no valid final score commitment exists after the grace period, the epoch is skipped with zero creation. No shortfall is owed, queued, extended or back-filled.

12.6.4 Distribution and Reward Maturation

For a valid epoch with $\widehat{S}_e > 0$, domain a is entitled to

$$R_{a,e} = \left\lfloor M_e \frac{\widehat{S}_{a,e}}{\widehat{S}_e} \right\rfloor, \qquad \sum_a R_{a,e} \le M_e.$$

A claimant supplies a Merkle proof, but payment always goes to the committed earning identity, never to the transaction sender. Integer remainder and unclaimed value stay unassigned in the epoch distributor; no operator may redirect them. The distributor snapshots an immediate fraction $b/10000$, a stream length n and maturation-epoch duration τ. If t_0 is the claim time, the matured amount at time t is

$$A_{a,e}(t) = I_{a,e} + \left\lfloor (R_{a,e} - I_{a,e}) \frac{\min\left(n, \left\lfloor \frac{(\min(t,t_f)-t_0)_+}{\tau} \right\rfloor\right)}{n} \right\rfloor, \qquad I_{a,e} = \left\lfloor R_{a,e} \frac{b}{10^4} \right\rfloor.$$

For an un-forfeited claim $t_f = +\infty$. On proven fraud, t_f is the forfeiture time and the unmatured remainder is permanently void; value already matured remains payable. Maturation is delayed payment for completed work and a period in which fraud remedies remain meaningful; it is not a return for passively holding or staking TOS.

12.6.5 Commit, Challenge and Native Settlement

The scoring and issuance pipeline separates expensive interpretation from deterministic consensus:

1. Public chain data yields settled-work rows. The interim feed records only settled Task Escrows and paid Service Actor requests; a production receipt adds capability class, units, explicit evidence level and price-cap inputs.
2. Independently implemented scorers apply the same frozen methodology, rate card and common-control inputs and must reproduce O_e, every $\widehat{S}_{a,e}$, $\widehat{S}_e$ and the Merkle root byte for byte.
3. A committer bonds the root, total score, organic settled value, methodology hash and rate-card hash. A symmetric bonded public challenge can present contrary evidence. An unreviewed challenge fails the root safe after its deadline; an absent or rejected root cannot halt later epochs.
4. After the full challenge window, the masterchain verifies the audited contract code hash, canonical address, approved reviewer, final status, economic tuple and frozen configuration. Collators and validators call one shared pure integer derivation for M_e; disagreement makes the block invalid.
5. The settlement actor deploys an audited per-epoch distributor. Claims are permissionless, Merkle-verified, pro rata, paid only to the committed identity and subject to the snapshotted maturation schedule.

Consensus does not decide whether an answer is intelligent. It decides whether the approved, challenge-complete score commitment and exact supply arithmetic authorize a bounded mint. This is an optimistic-verification boundary: its security depends on data availability, independent recomputation, a live challenger set and a reviewer that cannot be chosen by the committer. Verification of untrusted ML is itself an active research area; replication, proof-of-learning and cryptographic proof systems carry materially different cost and soundness trade-offs.[4]

12.6.6 Attack Economics and Launch Criterion

For a fraud strategy with maximum gain G, probability d of detection and successful challenge, slash or forfeiture loss L, and direct attack cost C, a conservative bound is

$$\mathbb{E}[\Pi_{\text{fraud}}] \leq (1-d)G - dL - C.$$

The economic launch criterion is not the existence of a bond; it is evidence from adversarial tests that the right-hand side is negative for the largest plausible single-epoch strategies, including wash settlement, payer-earner collusion, forged evidence, scorer equivocation, reviewer capture, address splitting, duplicate receipts and withholding evidence until after the challenge window. The project must publish assumed G, d, L, C, sensitivity ranges and the largest profitable counterexample found. Where reliable bounds cannot be established, the affected evidence multiplier, cold-start floor or emission coefficient must be reduced, possibly to zero.

The implemented repository contains the interim settled-work data plane; score-commitment, challenge, settlement and distributor actors; an independent Merkle implementation and claim path; governance-gated ConfigParams 90–93; a cumulative native-supply cap; and a shared collator/validator mint derivation. The `capAipow` feature is off in the genesis template. Production

[4] For one primary-system discussion of these trade-offs, see Gensyn, "Verde: a verification system for machine learning over untrusted nodes," 2025:
https://blog.gensyn.ai/verde-a-verification-system-for-machine-learning-over-untrusted-nodes/.

activation additionally requires the full settlement-receipt schema, a versioned scoring methodology and control-domain policy, two independently authored scorers that reproduce long-running shadow epochs, data-availability and challenge tooling, review of the reviewer/governance trust model, external contract and consensus audits, adversarial economic testing, and counsel review of then-current applicable law. None of the community allocation is created before those gates pass, and this section promises no completion date, reward rate, adoption level or token value.

12.7 Broad Distribution and Concentration Controls

The protocol avoids allocating the initial supply to founders, investors or a central treasury. Its validator distribution is intended to broaden through recurring elections and open network participation. Initial controls include:

- four equal-weight original validators rather than one bootstrap validator;
- equal and capped bootstrap funding to separately disclosed controlling wallets;
- recurring protocol elections that replace the zerostate validator set;
- a 10,000-TOS minimum and 10,000,000-TOS maximum submitted stake;
- a four-validator, 40,000-TOS aggregate protocol minimum;
- an initial maximum effective-stake factor of one, which keeps the minimum four-validator elected set equal in effective weight;
- a maximum of 400 validators and 100 masterchain validators;
- public disclosure of beneficial control, key custody, funding, hosting, network, geography and related-party relationships; and
- public concentration metrics for the largest one, four and ten validators and for known related parties.

Four validators are only the bootstrap minimum, not a decentralization claim. The recommended public-launch participation target is 64 independent operators, with a long-term target of at least 75 eligible validators.

A later increase in the effective-stake factor requires sustained independent participation, concentration analysis, election simulation and public governance review. It also has an arithmetic precondition that is not a matter of review. The Elector pays each elected validator on $\min(s_i, f \cdot s_{\min})$, where f is the effective-stake factor and $s_{\min}$ is the smallest stake in the elected set, and it refunds the remainder. The largest share of effective weight one entry can therefore hold in a set of n validators is

$$\frac{f}{f+n-1},$$

and requiring that this stay below one third gives

$$f < \frac{n-1}{2}.$$

The bound applies to the smallest set the configuration permits rather than to the set that happens to be running, because the Elector may install exactly the minimum in any round. Raising the factor to three therefore requires a minimum validator set of at least eight, and the minimum must be raised first: increasing the factor ahead of it would leave a window in which the configuration admits a set where one entry holds half the weight. Repository tooling enforces both the bound and the ordering, and refuses to emit a signable proposal for a combination that violates them.

These controls reduce initial administrative allocation and single-entry dominance, but no protocol can guarantee that market ownership or operational control will never concentrate. Stake-

proportional rewards may compound existing holdings, one controller may attempt to operate several identities, and early rewards necessarily go to the validators then elected. Broad ownership therefore also depends on new validator entry, service payments, voluntary transfers and transparent monitoring. TOS communications must describe these residual risks and must not claim guaranteed decentralization.

12.8 Supply Accounting, Burns and Transparency

Supply reporting distinguishes:

- **gross native creation**: every TOS created in zerostate or by finalized block creation, including amounts later burned;
- **outstanding supply**: gross native creation minus provably burned TOS;
- **circulating supply**: outstanding TOS excluding published system reserves, locked stake and other consistently defined non-circulating balances; and
- **validator block creation**: cumulative ConfigParam 14 creation after genesis.

Burning unused bootstrap funds or penalties reduces outstanding supply but does not erase historical gross creation. Public reporting must derive creation and burn totals from finalized chain data, not solely from wall-clock estimates. Before launch the project must publish every genesis balance, validator and controlling-wallet identity, relevant configuration value, zerostate hash and bootstrap transaction cap. During operation it must publish current reward parameters, cumulative creation, burns, circulating-supply methodology, trailing block rates, target-date projections, validator concentration and every monetary configuration proposal and vote.

12.9 Service Economic Flows

The architecture can use native TOS for:

- escrowed task budgets and agent payouts;
- refunds after rejection, cancellation or timeout;
- service-call settlement under explicit quotes and maximum charges;
- registry bonds and future verifier staking;
- defined discovery, relay, verification or storage services; and
- gas for messages and persistent actor state.

Economic activity is designed to arise from completed service transactions and explicitly defined infrastructure services, not from merely remaining online or declaring hardware capacity. Token ownership does not grant an agent unlimited automation authority, prove service quality or override terminal policy.

Service transaction volume, validator fees, service-provider revenue and token market value are distinct concepts. This paper makes no forecast or promise regarding any of them.

12.10 Governance, Communications and Participant Responsibilities

Ordinary authenticated configuration governance retains the ability to change block rewards, validator counts, stake limits, election timing and other configurable protocol parameters. Every monetary proposal should disclose its proposer, old and proposed values, activation conditions, expected supply impact and recorded votes. The ability to change configuration is a material governance risk and means neither the 500-million validator-creation target nor the five-billion total-supply target is a hard technical cap.

Public descriptions of TOS must be accurate, balanced and consistent with released code and accepted network configuration. They must separate implemented functions from plans, avoid

predictions of token value or return, disclose material technical and economic risks, and avoid presenting a policy target as a guarantee. No operator may describe validator compensation as risk-free interest or a reward available merely for purchasing or holding TOS.

The base protocol does not replace obligations that may apply to exchanges, custodians, wallet operators, payment intermediaries, token distributors or commercial service providers. Each participant is responsible for determining whether its activities require authorization, customer and counterparty checks, transfer records, transaction monitoring, tax reporting, sanctions controls, consumer disclosures, data protection or other jurisdiction-specific measures. Service-layer implementations should expose the records and policy boundaries needed for their operators to implement those controls without placing private customer data in public consensus.

12.11 Risk and Sustainability Disclosures

TOS may lose some or all market value. Transferability, exchange availability and liquidity are not guaranteed. Users face key-loss, custody, smart-contract, consensus, validator, governance, software, cybersecurity, service-provider, market and changing-regulatory risks. Network faults, concentrated stake, configuration changes or insufficient fee demand may impair operation or validator incentives. TOS is not redeemable for official currency by the protocol and is not presented as a deposit, insured balance or protected investment product.

Consensus uses stake-bonded validators rather than proof-of-work mining or a proof-of-work token giver. Validator nodes, networking and supporting infrastructure still consume energy and hardware resources. AI terminals and service workloads have separate resource impacts that must not be attributed to consensus alone. Material energy and environmental claims should be based on a published methodology and measured deployment data; no unverified claim of zero impact is made.

This section describes protocol design and policy rather than providing legal, tax or investment advice. Classification and participant obligations depend on the facts of an activity and may change as the network, services and applicable requirements evolve.

13 Implementation Status

This whitepaper separates repository implementation from architectural direction. “Implemented” means this repository contains the relevant node, contract, tooling or test foundation. It does not imply public commercial deployment, external audit, production service availability or adoption. “Partial” means a reusable primitive exists but the interoperable product surface does not. “Planned” identifies work assigned to a future protocol or product repository.

Capability	Status	Scope
Native TVM execution and actor messages	**Implemented**	Node, contracts, cells, gas and value-bearing messages
Masterchain and shardchain protocol	**Implemented**	Validator and full-node protocol stack
ADNL, DHT, RLDP, QUIC and TOS Sites	**Implemented**	Core transport and networking primitives
TOS DNS and resolver chaining	**Implemented**	Naming primitives; not a complete public registrar product

Capability	Status	Scope
Agent Wallet profiles	**Implemented**	Keys, policy, funding, activation, runtime binding and inspection
Agent Account	**Implemented**	Deterministic deployment, controller actions, limits and rotation
Task Escrow	**Implemented**	Claim, accept, reject, result, review, dispute, resolve and settlement
Agent and task read APIs	**Implemented**	Authenticated reads, filtering, timeout and error taxonomy
Capability Registry	**Implemented**	Per-actor entry, metadata hashes, bond, activity and reputation
Service Actor contract	**Implemented**	Access policy, pricing, rate limits, calls, responses and revenue
Dispute and Proof Attestation foundations	**Implemented**	Compact resolution and evidence commitments
AIPoW settled-work data plane	**Implemented**	Interim indexer feed for settled Task Escrows and paid Service Actor requests; full receipt fields remain planned
AIPoW commitment and distribution actors	**Implemented**	Bonded root challenge, epoch settlement, Merkle claims, maturation and forfeiture; not production-activated
AIPoW native issuance path	**Implemented**	ConfigParams 90–93, cumulative cap and shared collator/validator derivation; `capAipow` is off by default
Pooled-stake contract and tooling	**Partial**	Application-layer arrangement, not a protocol feature; contract, build lock, daemon support and end-to-end run exist; no user entry point and no economic effect at factor one
AIPoW production scoring and activation	**Planned**	Frozen methodology, control-domain inputs, independent scorer reproduction, audits, shadow epochs and governance gate
Public `.tos` registrar product	**Planned**	Ownership lifecycle, SDK, deployment and operator tooling
ARD compatibility profile	**Planned**	Pinned external version, catalog mapping, publisher binding, media types and conformance
TOS ARD Registry	**Planned**	Independent REST search, ingestion, federation, provenance, visibility and bounded crawling
TOS Service Protocol	**Planned**	Descriptors, sessions, quotes, receipts, evidence and conformance

Capability	Status	Scope
Edge Core and discovery clients	**Planned**	Terminal enforcement, SDKs, ARD search and verifiable TOS handoff
Managed AI terminal	**Planned**	Approved model services, bounded scheduling, metering and receipts
Physical AI terminal	**Planned**	Offline operation, safe updates, real-time priority, isolation and fleet management
Profile indexes and ranking	**Planned**	AI, storage and commerce enrichment without treating derived data as authority
Public-testnet hardening and external audit	**Planned**	Persistent deployment, load testing and independent review

14 Verification and Release Gates

No terminal profile is production-ready merely because its happy path works. Release suites must cover:

- canonical encoding, signature domains, replay, expiry, idempotency and unknown critical extensions;
- contract authorization, escrow conservation, cancellation, refunds and disputes;
- AIPoW score arithmetic, rate-card caps, duplicate rejection, common-control aggregation, deterministic Merkle roots and independent scorer equality;
- AIPoW commitment and challenge deadlines, reviewer failure, malformed roots, canonical-address and code-hash binding, skipped epochs and data availability;
- AIPoW demand-coupled pool arithmetic, integer overflow and rounding, cumulative supply cap, pro-rata conservation, maturation, forfeiture and zero-mint behavior;
- adversarial AIPoW wash settlement, payer-provider collusion, Sybil splitting, evidence forgery, scorer equivocation, challenger inactivity and reviewer capture;
- resolver, descriptor, quote, payment, invocation, receipt and restart flows on a multi-node TOS network;
- pinned-version ARD publication, `POST /search`, publisher verification, provenance, withdrawal, nested catalog and bounded federation behavior;
- malformed and oversized catalogs, decompression, redirect and federation loops, SSRF and DNS rebinding, prompt injection, stale rollback, equivocation, poisoned ranking and private-catalog leakage;
- adapter crash, timeout, disconnect, malformed input, out-of-memory and failed-payment cleanup;
- bounded RAM, accelerator memory, disk, cache, watcher, connection, retry and journal behavior under sustained load;
- physical real-time priority, network partition, stale policy, reconnect and idempotent reconciliation;
- signed update verification, compatibility rejection, power-loss activation, rollback and staged fleet abort;
- actuator isolation, local safety override, fleet revocation and bounded command fan-out.

Release gates include a threat model, protocol conformance vectors, reproducible builds, system-

service or container hardening, migration and rollback procedures, independent review of externally exposed and settlement-critical components, and observed testnet operation.

15 Roadmap

15.1 Cross-Cutting Gate: AIPoW Shadow Operation

Keep `capAipow` inactive while completing the full settlement-receipt schema, frozen scoring methodology, rate card, common-control inputs and public epoch dataset. Run at least two independently authored scorers over sustained testnet shadow epochs; publish every root, disagreement, challenge, concentration metric, attack simulation and parameter sensitivity. Activate native creation only after contract and consensus audits, adversarial economic review, reviewer/governance hardening and an explicit public governance decision. A failed or delayed gate creates no emission debt.

15.2 Foundation: TOS Core

Maintain consensus, native TVM execution, networking, wallet and reusable actor-contract foundations as a stable dependency. Application workloads and vertical product release cycles remain outside the validator.

15.3 Phase 0: Base Protocol and Edge Core

Specify canonical descriptors, manifests, authentication, quotes, payment authorization, receipts, evidence levels, profile negotiation and conformance vectors. Build the public `.tos` registrar, chain adapter, SDKs and Edge Core. Implement the pinned ARD compatibility profile, catalog publisher, independently deployable ARD Registry, bounded federation and deterministic multi-node reference harness.

15.4 Phase 1: Managed Inference Terminal

Deliver a Tier 1 Linux/NVIDIA reference terminal for provider-approved models and bounded AI services. Complete resource probes, initial runtime adapters, task admission, streaming, metering, signed receipts, restart recovery and resource-soak testing. This phase does not accept arbitrary consumer execution.

15.5 Phase 2: Site-Bound Physical Terminal

Deliver a Jetson/ARM reference profile with disconnected local execution, bounded offline authority, tamper-evident reconciliation, signed A/B updates, real-time priority, physical-I/O isolation, independent safety interlocks and staged fleet management.

15.6 Phase 3: Additional Service Profiles

Add storage, commerce, digital and physical fulfillment, human-service and controlled composition profiles without forcing their business state machines into the AI protocol or validator.

15.7 Phase 4: Advanced Network Services

Add production relays, prepaid or channel settlement, multi-region routing, replication, stronger attestation, verifier markets and policy-aware orchestration after the base protocol and terminal cleanup invariants are stable.

16 Non-Goals

TOS does not aim to:

- run private model inference directly in consensus;
- store prompts, credentials or large model outputs on-chain;
- rent bare GPUs or expose raw accelerator devices;
- accept arbitrary consumer-supplied containers, programs or models;
- expose a public shell, Docker socket or unrestricted host interface;
- put blockchain consensus in a hard real-time or safety-control loop;
- expose raw CAN, GPIO, serial, fieldbus or actuator access as a public service;
- reward a terminal merely for remaining online or declaring TOPS;
- hard-code one model provider, oracle, verifier or proof system;
- make third-party indexers authoritative for funds or permissions;
- replace ARD with a TOS-only general-purpose agent catalog or treat ARD discovery as authorization, live admission, payment or physical-control authority;
- give an agent runtime unrestricted owner custody by default;
- bundle unrelated execution engines into the production node.

17 Conclusion

TOS Core provides the implemented blockchain and networking substrate for persistent identity, bounded authority, asynchronous coordination and settlement. TOS Network is the broader product objective: an open service economy in which independently owned agents, models and physical terminals can transact without transferring custody of their hardware, data or operational policy to one platform.

The differentiation is not ownership of a particular accelerator or foundation model. It is the ability to determine which declared service may execute a request, under what limits, privacy and evidence policy, at what price, and how the result is receipted and settled. Physical terminals extend that model into real-world sites while preserving offline operation, deterministic local priority and independent safety authority.

AIPoW adds a bounded supply-side acquisition mechanism to that transaction layer. Its investment-relevant claim is measurable rather than rhetorical: unrelated settled demand must produce reproducible evidence-weighted score; issuance must remain a visible, capped function of that demand; and rewards must improve independent service supply without creating circular volume or control-domain concentration. If those metrics do not hold during shadow operation, the mechanism remains inactive or its parameters move toward zero.

This positioning is deliberately independent of foundation-model competition. TOS Network does not train models and does not need to be preferred over any other foundation-model provider, open-weight release or inference stack: its function is identical regardless of which model a participating node runs. As capable open-weight models in the 8B–14B range increasingly run on owner-operated edge hardware rather than exclusively in centralized data centers, described in Sec-

tion 1.4 and Section 1.5, the network's addressable participants shift from a comparatively small number of data-center operators toward a much larger population of owner-operated edge, regional and cloud nodes. TOS's role does not change with that shift: it remains the identity, discovery, authorization, evidence and settlement layer that lets each of those nodes be found, trusted and paid.

ARD makes these resources interoperably discoverable across the broader agentic ecosystem. TOS makes a discovered service safely actionable and transactable: it binds the resource to current authority, admission, execution policy, evidence, payment and settlement without turning a search registry into a trusted control plane.

The result is a focused positioning: TOS Network is being built as the open coordination and settlement network for AI services and physical edge intelligence – in short, an Internet Layer for Autonomous Agents spanning mobile, edge, regional and cloud compute tiers. The chain foundation exists; the interoperable service protocol and terminal products remain work to be specified, implemented, tested and independently reviewed.

Copyright and License